QuickBites

Transaction Reliability and Robustness Report

Module B Assignment

Date: April 4, 2026

Team Members	Role
Bhavya Parmar	Backend and Database Design
Surriya Gokul	SQL Indexing and Benchmarking
Sonagiri Thoshith Kautilya	Frontend and UI or UX

GitHub: https://github.com/mohit-codes-21/QuickBites

Video: View Submission Video

Indian Institute of Technology Gandhinagar

Contents

1 Introduction **2**

2 Verification Scope and Evaluation Criteria **2**

3 Implementation Controls for Correctness **2**

3.1 Transaction and rollback discipline 2

3.2 Concurrency and race hardening 2

3.3 Data integrity and observability 2

4 Experimental Setup and Test Scenarios **2**

5 Required Reliability Explanations **3**

5.1 How correctness of operations is ensured 3

5.2 How failures are handled 3

5.3 How multi-user conflicts are handled 3

5.4 What experiments were performed 3

5.5 Observations and limitations 4

6 Results and Analysis **4**

6.1 Scenario scale and created-entity counts 4

6.2 Suite-level outcomes 5

6.3 Detailed latest test entries 5

6.4 Stress and load comparison 6

6.5 Endpoint-level latency summary 6

7 Visual Evidence **6**

8 Discussion **9**

9 Reproducibility **9**

10 Conclusion **9**

1 Introduction

This report documents reliability verification for QuickBites under concurrent usage, race pressure, injected failures, and sustained load. The objective is to validate transaction behavior against ACID principles and assignment criteria: correctness of outcomes, rollback safety, multi-user isolation, and robustness under stress.

2 Verification Scope and Evaluation Criteria

The verification scope covers five suites: concurrent usage, race condition testing, failure simulation, ACID checks, and stress testing. Evaluation focused on four properties:

- **Atomicity:** operations either commit fully or rollback fully.
- **Consistency:** no invalid relational states, including orphan references or incomplete entities.
- **Isolation:** one user flow cannot corrupt or leak partial state into another.
- **Durability:** committed data persists across process restarts.

3 Implementation Controls for Correctness

3.1 Transaction and rollback discipline

Critical write paths execute in explicit transaction boundaries with commit-on-success and rollback-on-exception behavior. Failure tests intentionally inject partial-signup and partial-request conditions to verify that incomplete entities are not persisted.

3.2 Concurrency and race hardening

Race-sensitive operations are serialized using lock-backed techniques on shared resources such as customer carts and order assignment. Observed outcomes confirm one-winner behavior in race scenarios, with deterministic final state under contention.

3.3 Data integrity and observability

Database constraints and audit logs provide secondary integrity protection and post-run traceability. All suites emit structured metrics to CSV and JSONL for repeatable analysis and reporting.

4 Experimental Setup and Test Scenarios

Experiments were executed with Python test scripts and Locust against the local API service. The table below maps each suite to scenario intent and verification target.

Suite	Primary Script	Scenario Focus	Verification Target
Concurrent	`tests/concurrent_test.py`	Parallel reads and same-record writes	Isolation and non-interference
Race	`tests/race_condition_test.py`	Simultaneous signup and delivery acceptance	Single correct winner
Failure	`tests/failure_simulation_test.py`	Timeouts and forced disconnects	Rollback and no partial state
ACID	`tests/acid_verification.py`	Atomicity, consistency, isolation, durability	Property-level pass or fail
Stress	`tests/stress_test.py` and `locustfile.py`	High request volume and mixed CRUD	Correctness plus latency behavior

5 Required Reliability Explanations

5.1 How correctness of operations is ensured

Correctness is enforced through explicit transaction boundaries for critical writes, with commit on success and rollback on any failure path. Data integrity is further protected using relational constraints and post-operation validation checks. ACID verification scenarios confirm atomicity, consistency, isolation, and durability at the behavior level, not only at code level. Structured logs and metrics provide traceability so each claim can be tied to reproducible evidence.

5.2 How failures are handled

Failure handling follows a fail-safe pattern: if a write cannot complete safely, the transaction is aborted and rolled back. Timeout and forced-disconnect simulations verify that partial entities are not left behind. Mid-transaction fault injection during signup confirms that invalid partial users cannot authenticate afterward. This approach ensures the system prefers rollback over inconsistent partial commit.

5.3 How multi-user conflicts are handled

Multi-user conflicts are managed using serialization controls on race-sensitive resources (for example, shared cart updates and delivery acceptance). In signup races, only one request is allowed to succeed for the contested identity while competing requests are rejected or rolled back. In delivery acceptance races, one-winner behavior is preserved and duplicate assignment is prevented. These checks were validated under concurrent thread pressure and repeated contention.

5.4 What experiments were performed

The following experiments were executed for this report:

- Concurrent usage: four scenarios including 15 parallel reads, 15 parallel updates, delete-while-readers, and 25-way cart contention.
- Race conditions: 50 concurrent signup contenders and 10 concurrent delivery acceptance contenders.
- Failure simulation: three injected fault modes (mid-transaction rollback, timeout handling, forced disconnect).

- ACID verification: atomicity, consistency, isolation, and durability checks including restart durability.
- Stress testing: threading workload of 1000 requests across 40 threads and Locust headless run with 60 users (latest aggregate 3652 requests).

5.5 Observations and limitations

Observed behavior is robust: all latest test entries passed, and both stress modes reported zero request failures in the latest run set. The main performance limitation is elevated tail latency on write-heavy paths compared with read-heavy paths under contention. Additional limitations include local-environment benchmarking (single-machine, development server context), synthetic workload distributions, and the absence of multi-node deployment effects. Therefore, the results are strong for assignment-grade reliability validation but should be complemented with production-style load profiling for final capacity planning.

6 Results and Analysis

6.1 Scenario scale and created-entity counts

To make concurrency and race outcomes explicit, Table 1 summarizes how many users, workers, contenders, or requests were involved in each scenario, and what entities were actually created versus rolled back.

Scenario	**Load Scale**	**Created / Rolled Back**	**Observed Outcome**
Concurrent S1 (parallel reads)	15 reader threads	No new entities (read-only)	successful_reads = 15/15
Concurrent S2 (parallel updates)	15 updater threads	No new entities; one final item version	updates_ok = 15/15
Concurrent S3 (delete while reading)	14 readers + 1 deleter	No new entities; target record removed from active listing	readers = 14, post_visible = 0
Concurrent S4 (cart contention)	25 concurrent updates	No new users/items; cart quantity converged correctly	updates_ok = 25/25
Race signup	50 simultaneous signup attempts	Users created = 1, rolled back = 49	single active user row after race
Race delivery acceptance	10 delivery contenders	Winning assignments = 1	one-winner invariant maintained
Failure simulation suite	3 injected failure scenarios	Partial users/accounts created = 0	rollback preserved consistency in all 3 cases
Stress (threading script)	1000 requests across 40 threads	Temporary menu records created and cleaned = 187	error_rate = 0.000%
Stress (Locust headless)	60 simulated users, 3652 requests	Login sessions observed = 60, failures = 0	sustained mixed CRUD with 0 failures

Table 1: Scenario participation scale and created-entity outcomes from latest run artifacts.

These counts show that race-heavy user creation paths were stress-tested with high contention, while

rollback behavior prevented duplicate or partial writes.

6.2 Suite-level outcomes

Suite	Total	Passed	Failed	Pass Rate
acid	5	5	0	100.00%
concurrent	4	4	0	100.00%
failure	4	4	0	100.00%
race	2	2	0	100.00%
stress	1	1	0	100.00%

Overall latest status: **16/16** test entries passed (**100.00%**).

6.3 Detailed latest test entries

Test Case	Suite	Status	Duration (ms)	Parallel users/ops	Result Detail
ACID Atomicity	acid	PASS	749.684	1 client write flow	Signup rejected as expected after injected fault; no partial user visible.
ACID Consistency	acid	PASS	532.453	1 client, 4 write ops	Integrity checks show zero orphan orders, order-items, and customers.
ACID Durability	acid	PASS	1425.692	1 commit + restart verify	Committed record remained visible after controlled service restart.
ACID Isolation	acid	PASS	423.972	2 parallel actors	No dirty visibility observed during partial write and disconnect window.
ACID Summary	acid	PASS	3421.197	Aggregate of 4 checks	Aggregate ACID status: 4/4 checks passed.
Concurrent Scenario 1 - 15 parallel reads	concurrent	PASS	866.930	15 parallel readers	All 15 reads completed successfully under parallel execution.
Concurrent Scenario 2 - 15 parallel updates	concurrent	PASS	924.887	15 parallel updaters	All updates succeeded; final state converged to a valid latest write.
Concurrent Scenario 3 - delete while readers run	concurrent	PASS	887.844	14 readers + 1 deleter	Readers stayed stable while delete completed; post-delete visibility is zero.
Concurrent Scenario 4 - same cart item contention	concurrent	PASS	1045.663	25 parallel cart updates	Quantity updates serialized correctly; no corruption under 25 parallel writes.
Failure Simulation Summary	failure	PASS	2673.148	Aggregate of 3 tests	Failure suite aggregate status: 3/3 scenarios passed.
Failure Test 1 - Mid-transaction rollback	failure	PASS	790.622	1 signup operation	Signup aborted and rollback verified; account remains non-authenticatable.
Failure Test 2 - API timeout simulation	failure	PASS	937.499	1 timeout operation	Timeout handling preserved prior consistent state; no partial record written.

Test Case	Suite	Status	Duration (ms)	Parallel users/ops	Result Detail
Failure Test 3 - Forced disconnect mid-write	failure	PASS	907.160	1 partial-write sender	Connection interruption did not leave partially committed user/account rows.
Race Condition - 50 concurrent signup attempts	race	PASS	12979.271	50 concurrent contenders	Exactly one successful signup; remaining contenders rolled back cleanly.
Race Condition - concurrent delivery acceptance	race	PASS	1515.869	10 delivery contenders	Single delivery partner accepted order; one-winner invariant maintained.
Stress Test (threading fallback)	stress	PASS	21939.766	40 threads, 1000 ops	1000 requests, 0.000% errors, avg 870.285 ms, median 670.651 ms, p95 2471.012 ms.

6.4 Stress and load comparison

Metric	Threading Stress Script	Locust Headless Run
Total requests	1000	3652
Error or failure rate	0.000%	0.000%
Average response time	870.285 ms	380.101 ms
P95 response time	2471.012 ms	840.000 ms
Median response time	670.651 ms	310.000 ms
Throughput	45.579 req/s	58.775 req/s

6.5 Endpoint-level latency summary

Endpoint	Source	Avg (ms)	P95 (ms)	Error (%)	RPS
`GET /api/menu-items`	Threading	881.842	1422.253	0.000	14.038
`GET /api/menu-items?search=single`	Threading	317.413	453.526	0.000	12.762
`POST /api/menu-items`	Threading	1624.641	3653.763	0.000	8.523
`PUT /api/menu-items/<rid>/<iid>`	Threading	991.745	2976.474	0.000	4.968
`DELETE /api/menu-items/<rid>/<iid>`	Threading	843.913	2596.822	0.000	5.287
`DELETE /api/menu-items/<rid>/<iid>`	Locust	297.508	510.000	0.000	10.976
`GET /api/menu-items`	Locust	525.616	870.000	0.000	14.806
`GET /api/menu-items?search=single`	Locust	222.189	370.000	0.000	16.046
`POST /api/menu-items`	Locust	518.796	1400.000	0.000	10.622
`PUT /api/menu-items/<rid>/<iid>`	Locust	274.287	450.000	0.000	5.359

7 Visual Evidence

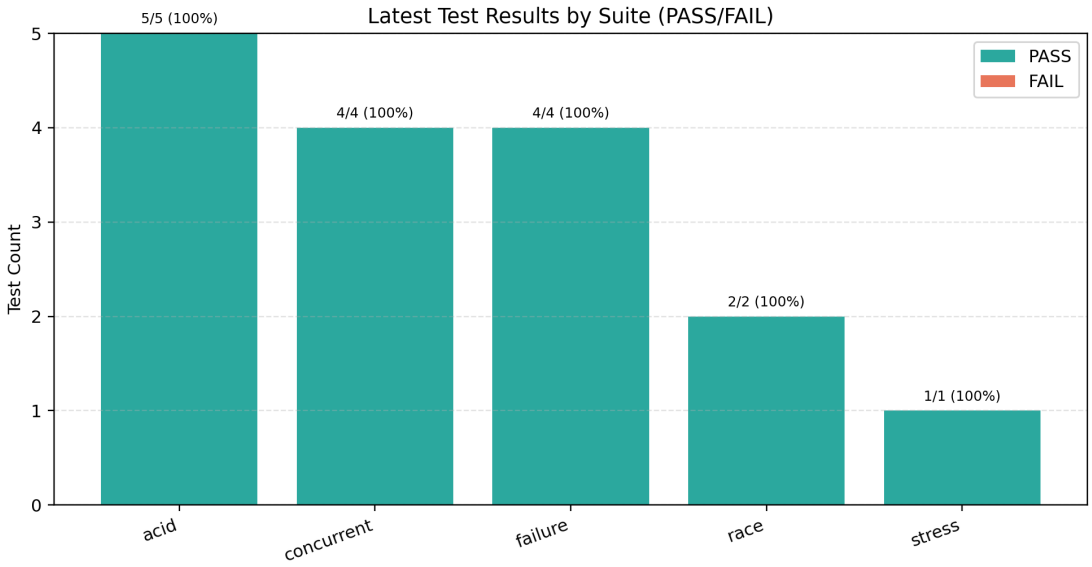

Figure 1: Figure 1: PASS and FAIL distribution by suite.

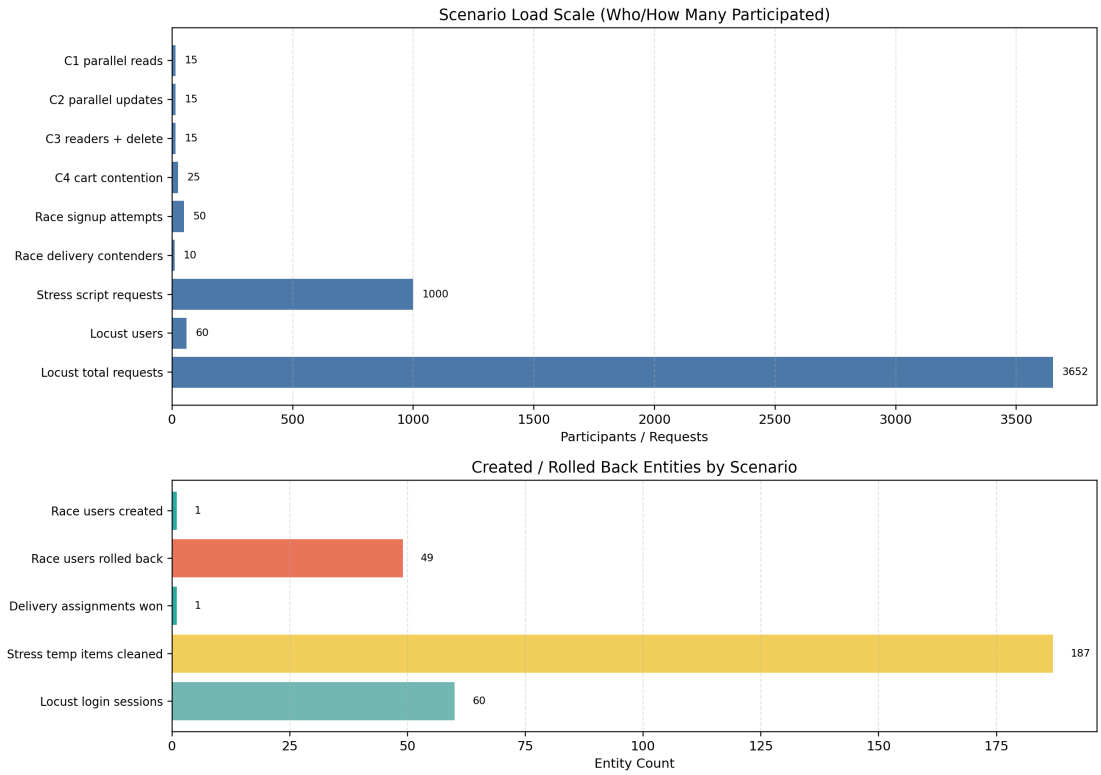

Figure 2: Figure 2: Scenario load scale and entity outcomes (participants, attempts, and created or rolled-back records).

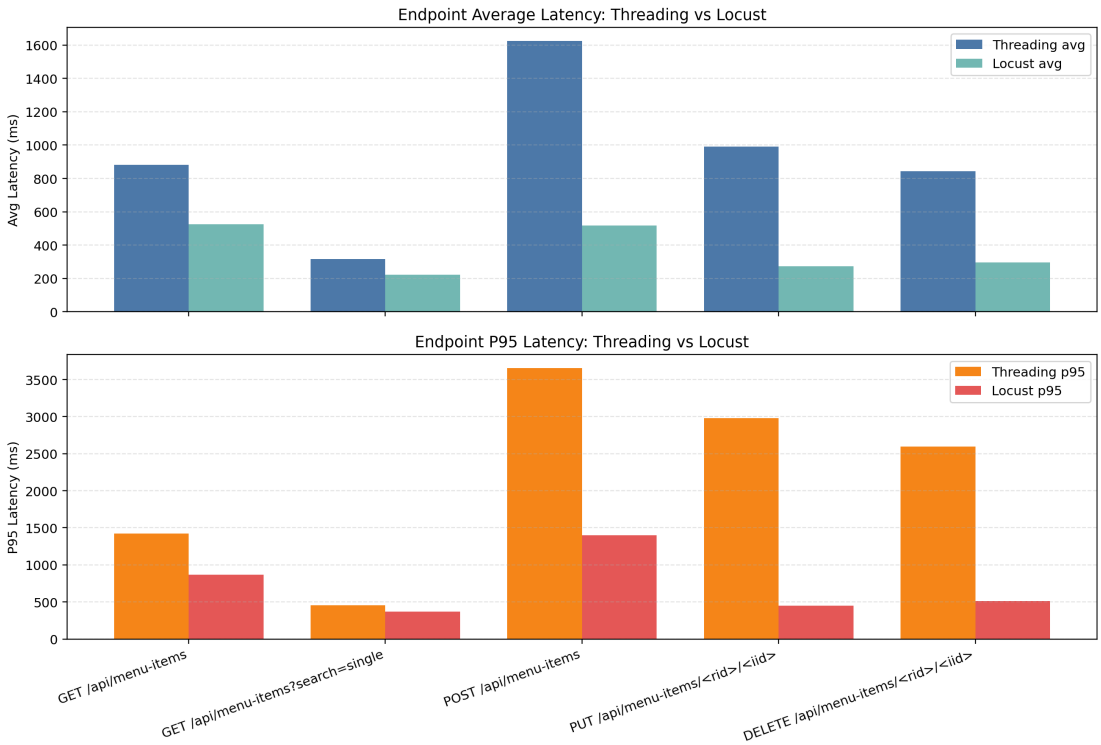

Figure 3: Figure 3: Endpoint latency comparison under stress (threading vs Locust, average and p95).

8 Discussion

The latest execution indicates strong transactional correctness across concurrent, race, failure, and ACID suites. All suites reported complete pass status with no residual integrity anomalies. Under stress, correctness remained stable (0% errors in both threading and Locust runs), but write-heavy operations exhibit materially higher tail latency than read-oriented operations. This pattern is consistent with lock contention and transaction serialization on critical write paths.

Locust appears faster than the threading stress script primarily because workload shape differs: Locust applies user think-time and distributed task pacing, while the threading script continuously drives requests with fixed worker concurrency and more sustained contention bursts. Therefore, Locust and threading values should be interpreted as complementary profiles, not direct one-to-one latency replacements.

9 Reproducibility

Latest verification can be reproduced with:

```
python tests/concurrent_test.py
python tests/race_condition_test.py
python tests/failure_simulation_test.py
python tests/acid_verification.py
python tests/stress_test.py
locust -f locustfile.py --headless -u 60 -r 20 -t 1m --host http://127.0.0.1:5001
```

To build this report via LaTeX:

```
make report-latex
```

10 Conclusion

The backend demonstrates required transactional reliability properties for this assignment: operations complete atomically, inconsistent partial states are prevented, concurrent users are isolated in critical workflows, and committed data survives restart conditions. System robustness under load is acceptable for assignment scope, with clear optimization targets identified around high-percentile write latency and lock-heavy update paths.